

Než začneš simulovat náhodu v Pythonu, zopakuj si, co je pravděpodobnost a relativní četnost, jak se pracuje s velkými čísly a poměry a jak v Pythonu funguje cyklus, seznam a funkce. Program pak poskládáš z věcí, které už znáš.

1. Velká čísla, mocniny deseti a poměry ZŠ

Kde to najdeš: Začátečníci H13 Odmocniny a zápis velkých a malých čísel, Začátečníci H8 Poměr a měřítko

Při simulacích se počty opakování píšou mocninami deseti: $10^4 = 10\,000$, $10^6 = 1\,000\,000$. Když se počet zvětší z 10^4 na 10^6 , je to $\frac{10^6}{10^4} = 10^2 = 100$ krát víc. Doby výpočtu porovnáváš poměrem: dělíš delší čas kratším. Pokud poměr časů odpovídá poměru počtu opakování, roste čas lineárně.

$$\frac{10^a}{10^b} = 10^{a-b} \text{ podíl mocnin deseti}$$

Příklad: Program pro 10^5 hodů běžel 0,08 s a pro 10^6 hodů 0,8 s. Roste čas lineárně?

- Počet hodů vzrostl $\frac{10^6}{10^5} = 10$ krát.
- Čas vzrostl $\frac{0,8}{0,08} = 10$ krát.
- Poměry se shodují → čas roste lineárně s počtem hodů.

Zkus sám:

- Kolikrát je 10^6 víc než 10^4 ?
- Zapiš 250 000 jako $a \cdot 10^n$.
- Program běžel pro 10 000 hodů 0,007 s. Kolik zhruba poběží pro 1 000 000 hodů, roste-li čas lineárně?

2. Pravděpodobnost a relativní četnost SŠ

Kde to najdeš: Začátečníci H29 Kombinatorika a pravděpodobnost na začátek, 1.+2. H53 Pravděpodobnost

Pravděpodobnost je teoretická šance, např. šestka na kostce má $P = \frac{1}{6} \approx 0,167$. Relativní četnost zjistíš z pokusu: počet příznivých výsledků vydělíš počtem všech pokusů. Při malém počtu pokusů se relativní četnost od pravděpodobnosti hodně liší, při velkém se k ní přibližuje – to je zákon velkých čísel. Odchylku spočítáš jako absolutní hodnotu rozdílu.

$$f = \frac{n_A}{N} \text{ relativní četnost} \quad |f - P| \text{ odchylka od pravděpodobnosti}$$

Příklad: Při 600 hodech kostkou padlo 112 šestek. Jaká je relativní četnost šestky a odchylka od teorie?

- $f = \frac{112}{600} \approx 0,187$.
- $P = \frac{1}{6} \approx 0,167$.
- Odchylka $|0,187 - 0,167| = 0,02$.

Zkus sám:

- Kolik šestek čekáš zhruba při 600 hodech kostkou?
- Ve 1 000 hodech padlo 170 šestek. Jaká je relativní četnost?
- Jaká je pravděpodobnost, že při hodu mincí padne panna, a jakou relativní četnost čekáš při 10 000 hodech?

3. Cyklus for a funkce range SS

Kde to najdeš: 1.+2. H49 Cykly a seznamy

Cyklus for opakuje stejný blok příkazů; range(n) dá čísla 0, 1, ..., n - 1, takže for i in range(10000) proběhne přesně 10 000krát. Zápis range(a, b, k) začíná u a, končí před b a jde po krocích k. Do cyklu patří jen to, co se má opakovat – výpis výsledku dej až za cyklus (bez odsazení).

Příklad: Program má sečíst čísla 1 až 5. Jak ho napíšeš a co vypíše?

1. s = 0 – součet začíná nulou.
2. for i in range(1, 6): s += i – range končí před 6, proto 6.
3. print(s) za cyklem vypíše 15.

Zkus sám:

1. Co vypíše: for i in range(2, 11, 3): print(i)?
2. Kolikrát proběhne cyklus for i in range(10000)?
3. Jaká čísla dá range(1, 7)?

4. Seznam jako počítadla SS

Kde to najdeš: 1.+2. H49 Cykly a seznamy

Seznam je řada hodnot s indexy od 0. Zápis [0] * 7 vytvoří sedm nul; indexy 1 až 6 pak můžeš použít jako počítadla pro čísla na kostce (index 0 zůstane nevyužitý). Když padne číslo h, zvětšíš počítadlo příkazem cetnosti[h] += 1. Funkce len vrátí délku seznamu a sum součet jeho prvků.

Příklad: Padly hody 3, 6, 6, 1, 3, 6. Co bude v seznamu cetnosti?

1. Začínáme s cetnosti = [0, 0, 0, 0, 0, 0, 0].
2. Každý hod h přičte 1 na index h.
3. Výsledek: [0, 1, 0, 2, 0, 0, 3] – šestka padla třikrát.

Zkus sám:

1. Kolik prvků má seznam [0] * 7?
2. Seznam s = [4, 1, 5]. Co je s[0] a co len(s)?
3. Jak zjistíš, kolikrát celkem se házelo, když máš seznam cetnosti?

5. Vlastní funkce: def a return SS

Kde to najdeš: 1.+2. H52 Podprogramy

Funkci definuješ slovem def, dáš jí jméno a parametry; výsledek předá zpět příkaz return. Hotovou funkci pak můžeš zavolat mnohokrát, třeba 10 000× v cyklu. Proměnné uvnitř funkce jsou lokální – zvenku je nevidíš, s okolím funkce komunikuje jen přes parametry a návratovou hodnotu.

Příklad: Napiš funkci prumer(s), která vrátí průměr seznamu, a použij ji na [2, 4, 9].

1. def prumer(s):
2. return sum(s) / len(s)
3. prumer([2, 4, 9]) vrátí 15 / 3 = 5.0.

Zkus sám:

1. Co vrátí funkce def dvakrat(x): return 2 * x pro dvakrat(7)?
2. Jaký je rozdíl mezi return a print uvnitř funkce?

Kde se to doučit

- **Nauč se Python – Užitečné funkce (modul random)** – Import modulu random, funkce randint a choice.
<https://naucese.python.cz/course/pyladies/beginners/basic-functions/>
- **Nauč se Python – Seznamy** – Práce se seznamy a počítání četností.
<https://naucese.python.cz/course/pyladies/beginners/list/>
- **JupyterLite – Python v prohlížeči** – Python bez instalace, stačí prohlížeč.
<https://jupyter.org/try-jupyter/lab/>

Výsledky ke kontrole

- **1. Velká čísla, mocniny deseti a poměry:** 1) 100krát; 2) $2,5 \cdot 10^5$; 3) asi 0,7 s
- **2. Pravděpodobnost a relativní četnost:** 1) asi 100; 2) 0,17; 3) 0,5; přibližně 0,5 (typicky 0,49–0,51)
- **3. Cyklus for a funkce range:** 1) 2, 5, 8; 2) 10 000krát; 3) 1, 2, 3, 4, 5, 6
- **4. Seznam jako počítadla:** 1) 7; 2) `s[0] = 4`, `len(s) = 3`; 3) `sum(cetnosti)`
- **5. Vlastní funkce: def a return:** 1) 14; 2) return předá hodnotu dál do programu, print ji jen vypíše na obrazovku